

杭州电子科技大学

本科毕业设计(论文)

(2024 届)

题目 语音驱动的三维人脸动画生成模型研究
与系统实现

学院 计算机学院

专业 计算机科学与技术

班级 20052313

学号 20051407

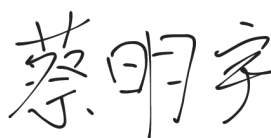
学生姓名 蔡明宇

指导教师 张聪

完成日期 2024 年 6 月

诚信承诺

我谨在此承诺：本人所写的毕业论文《语音驱动的三维人脸动画生成模型研究与系统实现》均系本人独立完成，没有抄袭行为，凡涉及其他作者的观点和材料，均作了注释，若有不实，后果由本人承担。

承诺人（签名）：

2024年6月3日

摘 要

随着虚拟数字人的发展和普及，音频驱动的三维人脸动画技术迎来了广泛深入的研究并应用于诸多领域。其基本流程为提取音频特征后，通过驱动人脸模型形成三维人脸动画。在近期的研究中，随着各种复杂生成式模型的引入，人脸动画的细节，情绪等复杂特征的表达逐渐成为了研究重点，但这些模型受限于大规模、高质量的数据集的缺失的同时，也带来速度上的损失。因此，本文在使用情绪潜代码思想的基础上，融合了基于条件扩散模型的轻量级网络，在尽量保留复杂特征的基础上提高运行速度，做到细节和速度的统一，同时在输出形式上，选择 **Blendshape** 参数，进一步提高速度的同时，也支持和不同的标准人脸相加得到最终 **Mesh** 序列。以三秒钟音频为例，该模型的预测耗时为 2.348 毫秒，平均平方误差在 0.00083 左右。

最后，在三维人脸模型的实现平台方面，本文选择在网页端实现，前端使用 **Vue** 框架和 **Three.js** 框架，后端使用 **Django** 框架，在更大众的平台上进行系统实现，降低用户的使用门槛。

关键词：音频特征提取；三维人脸动画；条件扩散模型；**Three.js** 框架

ABSTRACT

With the development and popularization of virtual digital humans, audio-driven three-dimensional facial animation technology has seen extensive and in-depth research and has been applied in many fields. The basic process involves extracting audio features and then driving the facial model to create three-dimensional facial animations. Recent studies have introduced various complex generative models, increasingly focusing on the expression of detailed features and emotions in facial animations. However, these models are limited by the lack of large-scale, high-quality datasets and also suffer from reduced speed. Therefore, this paper builds on the concept of emotional latent codes and integrates a lightweight network based on conditional diffusion models. This approach strives to preserve complex features while improving operational speed, achieving a balance between detail and speed. Additionally, by choosing Blendshape parameters for the output format, the model not only further enhances speed but also supports integration with different standard facial meshes to produce the final mesh sequence. Using a three-second audio clip as an example, this model's prediction time is 2.348 milliseconds, with a mean squared error of approximately 0.00083.

Finally, in terms of the implementation platform for the three-dimensional facial model, this paper opts for a web-based implementation. The frontend uses the Vue framework and the Three.js framework, while the backend employs the Django framework. This allows the system to be implemented on a more accessible platform, lowering the barrier to entry for users.

Keywords: audio feature extraction; 3D facial animation; conditional diffusion model; Three.js framework

目 录

1. 绪论.....	1
1.1 研究背景和意义.....	1
1.2 国内外研究现状.....	1
1.3 研究内容.....	3
1.4 论文组织结构.....	3
2. 相关技术介绍.....	4
2.1 音频特征提取.....	4
2.2 基本方法介绍.....	4
2.3 条件扩散模型.....	5
2.4 U-Net.....	5
2.5 TensorFlow.....	6
2.6 Vue.....	7
2.7 Django.....	7
2.8 三维模型.....	8
2.9 本章小结.....	8
3. 模型介绍.....	9
3.1 音频特征提取.....	9
3.2 特征分析网络.....	10
3.2.1 基本介绍.....	10
3.2.2 情绪潜代码.....	10
3.3 输出网络.....	11
3.3.1 扩散模型的前向、后向传播.....	11
3.3.2 改进的 U-Net 模型.....	12
3.3.3 音频特征条件注入.....	13
3.4 损失函数.....	13
3.4.1 位置偏差.....	13
3.4.2 连续偏差.....	13
3.4.3 正则化项.....	13
3.5 实验结果与分析.....	14
3.5.1 实验数据集.....	14
3.5.2 实验设置.....	14

3.5.3 实验指标.....	14
3.5.4 实验分析.....	14
3.6 本章小结.....	15
4. 系统设计与实现.....	16
4.1 需求分析.....	16
4.1.1 功能需求.....	16
4.1.2 界面需求.....	16
4.2 系统模块设计.....	16
4.3 系统流程.....	16
4.4 前端页面实现.....	17
4.4.1 音频文件上传.....	17
4.4.2 静态人脸模型展示.....	17
4.4.3 动态人脸动画实现.....	19
4.4.4 动画渲染问题处理.....	20
4.4.5 页面样式搭建.....	20
4.4.6 图标、弹窗美化.....	21
4.4.7 静态模型修改应用至动画.....	21
4.5 后端模型部署.....	22
4.5.1 部署模型概述.....	22
4.5.2 模型具体部署流程.....	22
4.5.3 跨域问题解决.....	23
4.6 系统测试.....	23
4.6 本章小结.....	24
5. 总结与展望.....	25
5.1 工作总结.....	25
5.2 不足和未来展望.....	25
致谢.....	26
参考文献.....	27

1. 绪论

1.1 研究背景和意义

语音驱动的三维人脸动画生成是计算机视觉和计算机图形学中一个非常有吸引力和实用性的研究课题。如 2021 年登上春晚的虚拟偶像“洛天依”，央视推出的虚拟主持人“小 C”等。其应用场景非常广泛，比如在服务行业实现虚拟客服、助手；在影视动画行业构造真实的虚拟角色动画；娱乐行业里实现虚拟偶像、直播皮套，游戏角色制作等等，以提高用户体验。虽然目前许多的虚拟人物模型有精美的建模和动作，但由于人脸运动的复杂性，用传统方法制作的面部动作缺乏真实感并且制作繁琐^[1]。

同时，伴随人工智能热潮，三维虚拟数字人也越来越普及，语音驱动的三维人脸动画技术作为高级的人机交互技术也被广泛采用。在诸多领域都有应用，例如提高虚拟代理人的真实性，辅助听力或语言障碍人士进行语言学习，大幅减少电影，动画领域的动画制作成本等等。

语音驱动的三维人脸动画生成的主要过程包括音频特征提取，视觉映射和最终的动画生成。各类生成式网络，包括 GANs, VAEs 和 Transformer 等等都能协助生成高质量的人脸动画，可以应用于诸多领域。但在具体应用时仍有以下几个问题：第一、对应语音的三维人脸模型由于成本高昂，很少有大规模的高质量三维视听数据库，在一定程度上限制了相关技术的发展；第二、涉及到情绪特征时，往往需要分离情绪和内容进行分析，提高了模型的负担，影响了实时性使用；第三、细节方面仍需要更加高效，便捷的解决方案。

基于以上问题，本文将探索一种以较小成本获取情绪特征，适用于小型数据库以及细节表达较完善的算法。

1.2 国内外研究现状

如今，语音驱动的三维人脸动画生成领域的研究主要分为两大方向：(1) Audio2Mesh, 即用语音直接预测 Mesh 序列信息；(2) Audio2ExpressionCoefficient, 即用语音预测得到的 Blendshape 参数和基本人脸 Mesh 做融合，得到最终的人脸动画。在过去，第一大方向在研究和应用中占据主导地位。然而，近年来随着 Blendshape 参数在三维模型中的广泛采用，基于第二大方向的研究和应用也迅速发展。这一类方法往往能基于基础人脸模型和输出的 Blendshape 参数序列生成高质量的人脸动画，可以通过替换基础人脸模型来支持泛用性，并且由于得到的结果是 Blendshape 参数而非实际的顶点信息，在类似于前后端分离的应用中也可以减少数据的传输量。特别的是，由于 Blendshape 参数可以理解为微表情的线性组合，

相比 Audio2Mesh 的方法，生成表情会相对更加自然。

2017 年，Nvidia 提出的语音驱动三维人脸动画模型 Audio2Face^[2]提出了一种端到端的卷积网络，从输入的音频直接推断人脸表情对应顶点位置的偏移，并在它原平台的 Omniverse Audio2Face 应用中被使用。而由于相同的音频信息可以对应非常不同的面部表情，该模型的做法是添加一个辅助输入，即情绪状态潜变量，并能将其作为一个用户输入的控制参数，从而输出不同情绪下的人脸动画。

2019 年，马普所在 CVPR 上提出的 VOCA^[3]模型侧重点则在泛用性上，其使用 DeepSpeech 提取音频特征，搭配 FLAME 人脸模型的同时，创新性地引入了身份信息，用于控制不同的说话风格和不同的说话人的搭配组合，并能将风格当作一个可选项，使用时间卷积和控制参数从任何语音信号和静态角色网格生成逼真的角色动画。缺点则是没有把情绪导入模型以及主要动作停留在嘴部附近。泛用性来说，Imitator^[4]进一步地用广义自回归 Transformer，以适应不同的说话风格，类似地，Faceformer^[5]基于 Transformer 来获取上下文的音频信息，并同样以自回归的方式得到连续的面部运动，但仍然无法较好地体现表情；Codetalker^[6]提出了一种基于离散运动先验的时间自回归模型，同样得到很好的面部连续性，但也有相同的质量问题。

国内近几年也有优秀的研究成果。2022 年，FACEGOOD（量子动力）开源的 Voice2Face 模型在 Audio2Face^[2]的基础上将输出从 Mesh 偏移变成了 Blendshape 参数，并把输出的 116 维 Blendshape 参数映射成 52 维的 ARKIT 系数，使用增强现实技术增强真实性。

同时，情绪的表达一直也是相关问题的一大难点，其根本原因在于音频与情绪的关联较小。情绪特征的引入也会导致具体内容的识别困难，因此主流的情绪表达方法是内容和情绪进行分离。一些方法包括 Audio2Face^[2]和 Videoretalking^[7]将情绪参数作为一个编辑可选项，在一定程度上解决情绪问题的同时也提高了情绪使用的灵活性。有一些新的网络提出了新的思路，比如 Emotalk^[8]提供了一种分开提取音频内容和情绪的情绪解缠编码器，可以使动画更明显地感受情绪信息，但也因为依赖于大规模的音频预训练模型，阻碍了实时性应用；吴昊哲等人^[9]说明了复合和区域性质分别对人脸动画生成的影响，并提出了基于非自回归主干结构的模型，思想与类似。

对于数据库稀缺问题，Speech4Mesh^[10]基于 DECA 低成本地构造足够且多样的 4D 数据，再基于 Faceformer 进行训练，以实现更逼真的效果。

近几年，扩散模型迅速兴起，逐渐应用于各种图像生成任务^[11-13]。Facediffuser^[14]作为第一次将扩散模型引入语音驱动的三维人脸动画的项目，使用基于 GRU 的扩散模型，并且同时支持直接 Mesh 或 Blendshape 参数的输出方式，提高了灵活性。进一步地，由于扩散模型等一类生成模型在生成细节较好的同时，往往需要较大数

数据集，而人脸动画生成领域的数据集一直较为缺乏，3DiFace^[15]提出了一维卷积网络的扩散模型，以提高模型在高质量的小数据集上的性能表现。

1.3 研究内容

本文的主要研究内容如下：

（1）广泛研究语音驱动的三维人脸动画生成模型的相关算法，并了解原理。

（2）基于相关优秀技术提出具有创新性的算法。本文使用修改过的条件扩散模型提高在小规模数据集下的三维人脸动画的细节表现，同时引入了情绪潜代码，以较低成本获取情绪特征。

（3）系统实现。使用 Vue 框架和 Three.js 框架搭建页面，Django 框架进行模型部署。系统支持传入音频文件，并根据音频文件生成对应的人脸动画，同时也支持 Blendshape 参数的查看和基础人脸模型的替换。

1.4 论文组织结构

本文共有五章：

第一章为绪论，介绍了人脸动画生成技术的背景意义，以及国内外相关技术的研究现状，并简要介绍了本文的主要研究内容。

第二章为相关技术介绍，模型方面具体有音频特征提取，相关基本算法，条件扩散模型，U-Net 网络；工具方面主要包括 TensorFlow，Vue 和 Django 框架和三维模型的使用。

第三章为算法模型的介绍，说明了模型的具体结构和引入情绪潜代码和条件扩散模型的优势，并对比了本文改进的算法模型和原先模型的实现效果。

第四章为系统实现。介绍了系统的基本组成和大致流程，以及各模块的具体制作和功能。

第五章为总结与展望，简要说明了本文的工作和不足，以及展望人脸动画生成领域的未来发展方向。

2. 相关技术介绍

2.1 音频特征提取

音频信号是一维的时间序列数据，通常由振幅随时间变化的波形表示。为了进行分析和处理，需要将其转换为具有语义信息的特征向量。常见的有时域特征，频域特征和时频域特征。在语音驱动的人脸动画生成领域，使用较多的包括 MFCC，W2V2，LPC。梅尔频率倒谱系数（Mel-Frequency Cepstral Coefficients，MFCC）是一种常用的音频特征提取方法，通过模拟人耳对声音的感知特性，将频谱信息转换成一组能够描述音频特征的系数。这里涉及到倒谱图（Cepstrum）的概念。

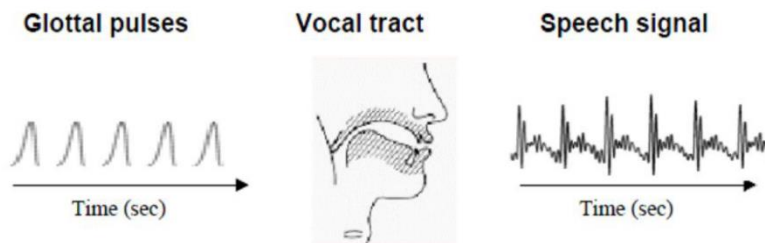


图 2-1 发音的基本过程

想象最开始通过声门的气体成为一种信号，称为 **Glottal pulses**，由声带和声道形成的滤波器输出语音信号（**Speech signal**）。这里由于声道具有随形状和大小变化的共振频率，使语音信号也出现共振峰。基于傅里叶变换和离散余弦变换，就能得到反应基频和共振峰的倒谱系数。一般使用前 12 个系数，再拼接一个当前 frame 的能量，共 13 个系数。线性预测编码（**Linear Predictive Coding, LPC**）是另一种常用的音频特征提取方法。LPC 提取特征的主要流程包括分帧，自相关分析，Levinson-Durbin 递推和提取特征，这里的特征可以选择 LPC 系数或是上文提到的倒谱系数。

在近几年，也出现了不少基于深度学习的特征提取方法。**Wav2Vec** 和 **Wav2Vec2** 是在人脸动画生成领域中使用比较多的方法。**Wav2Vec** 采用了称为“掩码语言建模”（**Masked Language Modeling, MLM**）的自监督学习方法，并使用多层卷积神经网络来提取音频的特征表示，最后通过预训练和微调后应用于具体任务。在此基础上，**Wav2Vec2** 引入了多方面的创新，包括引入了“随机掩码”（**Contrastive Predictive Coding, CPC**）方法，采用半监督微调 and 基于多层次对时间上下文的方法进行特征提取。

2.2 基本方法介绍

语音驱动三位人脸动画的算法可以基于传统的机器学习或深度学习方法。机器学习大部分采用隐马尔可夫模型（**Hidden Markov Model, HMM**）和高斯混合模

型（Gaussian mixture model, GMM）。多种方法也在基于此两种方法的基础上做改进，但仍有较大的局限性。HMM 训练时的计算量较大的同时，对较复杂的上下文依赖关系分析能力较弱；GMM 无法改变训练数据的内在结构，导致了无法跨数据库训练。深度学习有利于建立非线性映射，因此渐渐纳入选择范围，这一类方法五花八门，从一些经典方法包括深度卷积神经网络，LSTM 和 CRNN 等到一系列生成式模型，例如 GANs, VAEs, 扩散模型等等。引入深度学习模型可以进行端到端的学习，较好地处理非线性映射关系，提高生成质量以及进行大规模地并行处理，为各种应用场景带来更加逼真和自然的用户体验。

2.3 条件扩散模型

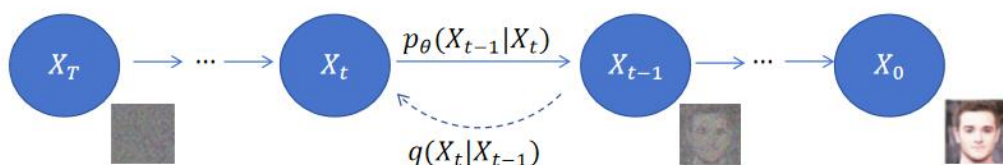


图 2-2 扩散模型的去噪过程

扩散模型作为生成式网络的新潮流，在生成图像和视频样本方面取得了一定的成功，特别是在处理高分辨率图像和长序列视频时具有一定的优势，在图像合成、图像修复、超分辨率重建等领域都有广泛的应用。扩散的概念基于非平衡热力学的概念，一句话来讲就是来自数据分布的样本通过扩散过程逐渐噪声，然后神经模型学习逐渐去噪样本的相反过程。其基本原理包含前向传播和反向去噪过程。前向传播过程向图像逐步添加高斯噪声，最终得到纯噪声，而反向去噪过程从纯噪音开始逐步去噪，最终得到真实图像。扩散模型学习的是从纯噪声生成图像的方法。希望为上述过程添加条件影响生成结果时，便引入了条件扩散模型（Conditional Diffusion Model）。

条件扩散模型是一种用于建模数据分布的概率生成模型。它是一种生成对抗网络（GAN）的变体，旨在通过学习数据的条件概率分布来生成具有特定条件的数据样本。在传统的生成对抗网络中，生成器网络从随机噪声中生成数据样本，而判别器网络则尝试区分生成的样本和真实数据样本。而在条件扩散模型中，生成器网络不仅接收随机噪声作为输入，还接收一个条件向量，用于指定所要生成样本的条件。这个条件向量可以是任何形式的附加信息，如类别标签、属性信息等。

2.4 U-Net

U-Net 主要用于图像分割领域。他用滑动窗口提供像素周围的区域（patch）作为输入来预测每个像素，非常适合图像数据集较少的情况。它的名字来自于其对称的编码器和解码器结构，形如字母“U”。U-Net 的核心思想是通过编码器部分逐步提取图像的抽象特征，并通过解码器部分将这些特征图与原始图像的局部信息

相结合，从而实现对图像进行像素级别的分割。

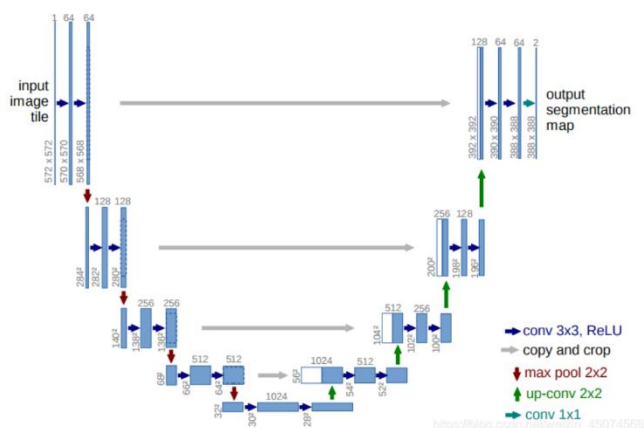


图 2-3 U-Net 结构框架

其结构如上图，U-Net 的编码器部分由多个卷积和池化层组成，用于提取图像的抽象特征。而解码器部分则包含对应的上采样和跳跃连接操作，用于将编码器提取的高层次特征与原始图像的低层次信息相结合，以实现精确的分割。此外，U-Net 还采用了一种称为“跳跃连接”的机制，即将编码器中的特征图与解码器中对应的特征图进行连接，以帮助网络更好地保留和传递重要信息。

大部分的扩散模型都使用 U-Net 作为基本的网络框架。主要得益于以下几点：

(1) U-Net 的“U”形结构适用于对图像进行精确的像素级别的分割，这正是扩散模型所需要的。(2) U-Net 的编码器，解码器结构可以帮助扩散模型有效地捕获图像中的上下文信息，从而提高了模型的性能；(3) U-Net 的结构较为直观且简单，具有较高地灵活性和扩展性，可以自由地搭配具体项目进行修改。

2.5 TensorFlow

TensorFlow 是一个开源的机器学习框架，由 Google Brain 团队开发并维护。它提供了一个灵活而强大的平台，用于构建和训练各种机器学习模型，从简单的线性回归到复杂的深度神经网络。TensorFlow 在深度学习领域得到了广泛的应用，成为了业界和学术界的首选工具之一。TensorFlow 可以参与模型的一系列过程，包括构建，训练，保存和加载，评估，加速提高性能，部署以及对模型数据进行可视化。其优点主要包括可移植性强，有良好的社区生态和内置算法，比较适合工业生产环境。

TensorFlow 和 PyTorch 一直是最受欢迎的深度学习框架，二者也有许多区别。简单来说，PyTorch 适合学术研究，TensorFlow 适合工业应用。TensorFlow 使用静态计算图，先构建后计算，提高了 TensorFlow 框架的性能；而 PyTorch 使用动态计算图，较为灵活，使得 PyTorch 更适合动态模型和实验研究，不过在 TensorFlow2.x 开始默认使用 Eager Execution，使其具有类似 PyTorch 的动态计算图功能。此外，TensorFlow 的生态系统较为完善，在模型部署方面，提供了一整套工具和框架，例

如 TensorFlow Serving, TensorFlow Lite (用于移动和嵌入式设备), TensorFlow.js (用于浏览器运行), 而 PyTorch 在这方面的生态系统还不够完善。

2.6 Vue



图 2-4 Vue 基本原理

Vue 是一套用于构建用户界面的渐进式前端框架, 其下框架有 Vue-Router, Vuex 等等框架。其设计初衷是为了简化开发者构建现代 Web 应用的流程, 提供了一套简洁、灵活且高效的工具和技术。其主要优点包括 API 设计简洁, 响应式数据绑定, 组件化开发, 使用虚拟 DOM 提高渲染性能等等。

Vue 是典型的 MVVM 框架, 也就是使用数据驱动视图和双向数据绑定。Vue 会自动监听数据的变化, 从而随之渲染页面结构; 在表单填写方面, 双向数据绑定可以自动将用户填写内容同步至数据源, 较大程度上减少了开发者的工作量。

在如今的前端开发中, 框架使用已成为基本开发方式。开发者会使用 Vue 或 React 作为基础框架, 搭配各种其他框架, 例如用 Element UI 库美化页面或 Three.js 支持动画和三维模型的展示等等。框架使用在简化开发者工作的同时, 也带来了前端技术的快速发展。

2.7 Django

Django 是一个用于构建 Web 应用程序的高级 Python Web 框架, 提供了许多开箱即用的功能。Django 遵循 MTV (Model-Template-View), 依次为定义数据结构和数据库操作的模型, 定义用户界面呈现方式的模板以及 与模型交互, 传递数据的视图。Django 提供了丰富的功能和工具, 包括 ORM (对象关系映射)、表单处理、认证、会话管理、缓存、国际化等。

Django 框架的优势主要体现在以下方面。Django 允许使用 Python 而不是 SQL 查询来操作数据库, 使得数据库操作更加简单; 提供了自动化的管理后台, 自动构建基于数据的管理界面; 提供了强大的模板系统构建 HTML 页面, 提高了开发效率; 拥有优秀的文档和大规模的开发社区; 可以搭配深度学习网络, 不用特别操作即可简单部署模型。

总的来说, Django 是一个功能强大、灵活易用的 Web 应用框架, 适用于开发各种规模的 Web 应用, 无论是小型网站还是大型企业级应用, 都能够快速高效

地实现。

2.8 三维模型

三维模型是计算机图形学领域的重要概念，用于描述物体的形状、结构和外观等信息。它是由一系列的顶点、边和面组成的几何数据集合，可以在计算机图形软件中进行创建、编辑和渲染，在电影和游戏制作中，三维模型被用来创建虚拟场景、角色等，实现逼真的视觉效果；在工程设计中，三维模型被用来进行产品设计和建模，帮助工程师进行设计评估和工艺优化；在医学领域，三维模型被用来模拟人体器官、病变和手术过程，辅助医生进行诊断和治疗。

三维模型针对不同实际场景，有多种类型，不过最常用的是多边形网格模型（Polygon Mesh Model），由许多连接的三角形或四边形组成，可以以较小的文件尺寸存储大量的几何信息。为了更好地支持三维动画，特别是人体或动物领域，引入了 Blendshape 参数。

通过在模型表面添加一系列 Blendshape 参数，可以通过改变这一系列权重去实现形变和动画效果。引入该项技术有以下几点优势：（1）可以更精确地控制表面形状，以实现更加细致和丰富的动画，类似于面部表情；（2）相比于直接控制顶点，控制 Blendshape 权重可以实现更平滑的形状过渡，使动画更加流畅；（3）Blendshape 较为直观和灵活，可以更方便地供设计师使用；（4）通过控制参数而非直接控制顶点可以减少处理的数据量，减少动画制作的复杂度和耗费资源。

2.9 本章小结

本章主要介绍了语音驱动三维人脸动画模型的发展和相关技术，以及本文使用的条件扩散模型。最后，还介绍了系统实现需要的相关技术和工具。

3. 模型介绍

现有的方法大部分使用 Transformer 等类似的生成网络，依赖于大型数据库的同时受限于高质量三维视听数据库的稀少；情绪表现的好坏依赖于情绪和内容的分离策略，现有的方法大部分使用大规模的预训练模型进行情绪和内容的分离，并分开分析。本文创新性地提出了融合情绪潜代码思想和条件扩散模型的新算法，在减少情绪特征的训练负担的同时，修改扩散模型，以提高在小数据集的性能表现和细节表达。模型的输入为音频窗口，而输出为对应这段音频窗口的中间帧的 Blendshape 参数。

网络训练的具体流程为特征提取，特征分析，情绪潜代码引入，条件扩散模型生成最终序列。本文提出的 Audio2Blendshape 模型主要可以分为三层网络：特征提取网路，分析与情绪引入网络和输出网络。

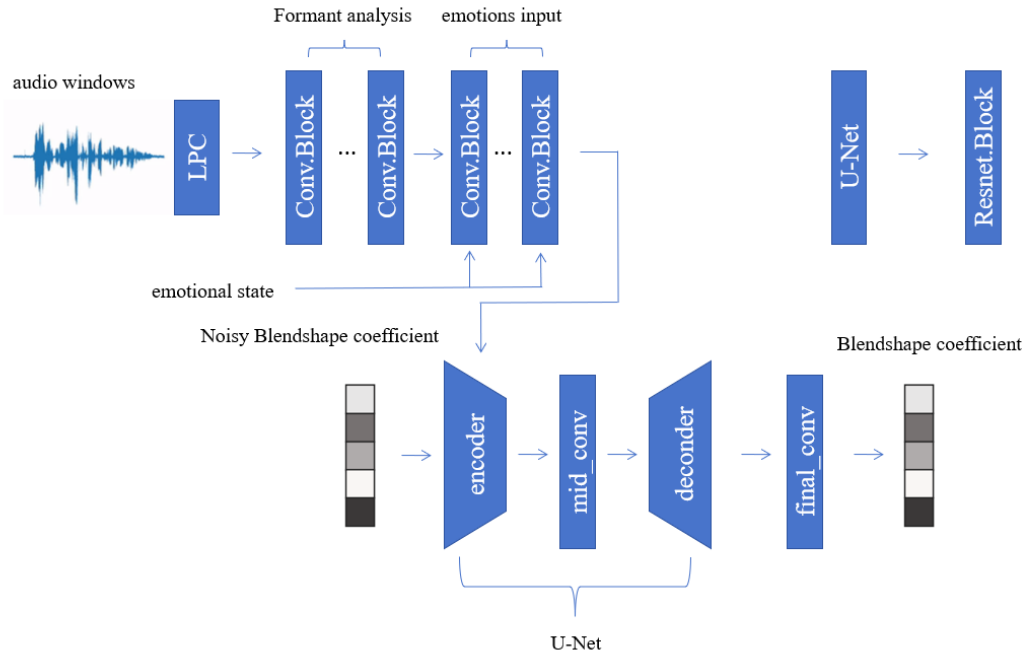


图 3-1 Audio2Blendshape 模型

本文的主体模型如上图，其中 U-Net 主要基于残差块构建，作为条件扩散模型的骨干网络。

3.1 音频特征提取

在正式提取音频特征前，首先将输入音频转至 16kHz 单声道，并对音量进行归一化处理，最后分割成窗口输入网络。本文使用线性预测编码（LPC）进行特征提取，通过共振峰重点关注音频的音素内容信息，而一定程度上忽略激发信号，也就是音频的音色等独特特征，以提高对不同人声的泛化性，此做法也在一定程度上

削减了情绪特征对于内容的影响。

LPC 认为语音取样的现在值可以由若干个过去值来逼近，即用过去的 p 个样本点来预测当前值：

$$\tilde{x}[n] = \sum_{k=1}^p a_k[n-k] \quad (3-1)$$

其中 a_k 表示为预测器系数。最后通过实际语音取样和预测取样之间的差值平方和去绝对唯一的一组预测器系数：

$$E_m = \sum_n e_m^2[n] = \sum_n (x_m[n] - \tilde{x}_m[n])^2 = \sum_n (x_m[n] - \sum_{j=1}^p a_j x_m[n-j])^2 \quad (3-2)$$

即：

$$J = E[e^2(k)] = E \left[(s(k) - \sum_{p=1}^p a_p s(k-p))^2 \right] \quad (3-3)$$

在完整过程中，每个短帧里，LPC 根据自相关系数预测其线性滤波器也就是声带的系数，并通过逆滤波提取激发信号。因线性的高度相似性，具体而言自相关系数较大程度上代表原始信号的压缩版本，本文跳过诸多步骤，直接用其自相关系数来代表其共振峰信息。这种做法不仅提高了特征提取速度，也较合适卷积神经网络去计算音频的瞬时功率。

具体而言，首先把音频分成 520 毫秒的窗口，也就是说假设当前这一帧会影响过去的 260ms 和未来的 260ms，将每个窗口又分为 64 帧，为每一帧保留了 32 位的自相关系数，作为每段音频窗口的提取特征。值得注意的是，虽然更少的自相关系数已经足以体现其共振峰信息，但为后续的特征分析，本文保留了一部分额外信息，所以最终得到 32 位。

3.2 特征分析网络

3.2.1 基本介绍

该部分分为两步：（1）进一步处理共振峰信息；（2）引入情绪潜代码。

由上一步得到的音频特征主要有三个维度，分别可以理解为抽象特征，时间轴以及共振峰轴。本部分需要将音频特征转换成合适作为条件引入条件扩散模型的形状。本文首先用五个带跨步，且核为 $1 * 3$ 的二维卷积神经网络降低共振峰轴的维数，同时增加抽象特征的维数；第二部分用类似的方法降低时间轴的维数，最终得到一维的高级抽象向量。

特别地，在第二部分拼接一个 E 维的潜在向量，这里将它称为情绪潜代码，作为数据样本的情绪特征。 E 需要选择合适的数量，太少会导致无法表示足够清晰的情绪状态，太多会导致训练负担以及过于契合对应的样本数据从而失去泛用推断的能力。本文最终选择 16 维作为比较合适维数。

3.2.2 情绪潜代码

语音情绪识别（SER）是指通过一段语音的特征来识别说话人的情绪状态，这一特征往往与语音的内容信息和语种无关。进一步地，可以分为离散式情绪描述方式和连续式情绪描述方式：离散指的是将情绪描述为离散类别，类似于愤

怒、快乐、惊奇等等形容词标签的形式；连续式将情绪描述为多维情感空间，每一维对应一个情绪的复杂属性，以二维空间来举例，可以分为情感的激烈程度和情感的正面负面程度。前者简单直观，模型也较简单，适用于需要实时性或只需要情绪大致类别的应用场景；后者情绪丰富，模型较复杂，高质量的数据集需求也较高，适合需要对情绪进行细致分析和建模的场景。

在语音驱动的三维人脸动画领域，情绪的识别又有新的难点。第一，若采用主流的情绪识别方式，需要为三维视听数据集进一步添加情绪标注，而三维视听数据集本就因为高成本导致难以构建大规模和高质量的数据集，相当于更加提高了数据集的要求，同时情绪标注的正确性和具体性还有待考证，尤其是情绪分类较复杂的情况；第二，语音驱动的三维人脸动画需要对其语音内容进行分析，这一部分特征与情绪特征有一定冲突。专注于情感表达的相关算法大都使用情绪特征和语音内容特征分开分析的方式，因此引入了复杂的预训练模型提取特征，影响模型预测的速度。而情绪潜代码可以在一定程度上解决这一问题。

本文会向网络引入辅助输入，也就是所谓的情绪潜代码。这些少量的情绪代码并不会直接从音频中提取，而是会在训练的过程中自动与每个样本数据关联起来。这在推理时就省去了复杂的特征提取过程，虽然会必然地损失一定的情绪信息，但能较好地提高运行速度。除了自动推断输入音频的情绪潜代码，在推理时作为辅助输入也是非常重要的应用，可以通过将不同的情绪潜代码和指定的输入音频轨道相结合，对于生成动画非常强大的控制手段。在未来，情绪潜代码也可以融合连续式情绪识别的思想，将多维的情绪潜代码映射成为多维的情绪空间，具体举例来说类似于调色盘，可以自由选择情绪的具体点位，以支持复杂的情绪选择。

本文将情绪状态表示为一个 16 维的潜在向量，在开始训练时初始化为高斯分布中的随机值。最终每个训练样本都会有这样一个潜在向量，存储起来成为情绪数据库。而这个情绪状态被拼接到了特征分析网络的每一层网络中，成为了损失函数计算图的一部分，也成为了能够被训练的参数。

3.3 输出网络

本文的输出网络选择经过修改的条件扩散模型。经过修改，该条件扩散模型可以适配本文的数据形状，同时提高在高质量的小型数据集上的表现。

3.3.1 扩散模型的前向、后向传播

扩散模型启发点来自非平衡热力学，系统和环境之间有着物质和能量交换，比如在一个盛水的容器中滴入一滴墨水，最终墨水会均匀的扩散到水中，但是如果扩散的每一步足够小，那么这一步就可逆。

其前向过程如下：

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I) \quad (3-4)$$

其中 $\mathcal{N}$ 为正态分布， $q(x_0)$ 是真实数据， t 代表向数据添加噪声的每一个 step，根据标准正态分布，此式也可改写为：

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon \quad (3-5)$$

上述过程中给出了每一步的前向传播过程，根据推导，可以得到直接从 X_0 到 X_t 的前向过程：

$$\alpha_t = 1 - \beta_t \quad (3-6)$$

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s \quad (3-7)$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad (3-8)$$

后向过程即求解 $q(x_t|x_{t-1})$ 的逆过程 $p(x_{t-1}|x_t)$ ，需要使用神经网络求解。同时，反向的条件概率分布也是高斯分布，那么网络需要计算的就是：

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (3-9)$$

最终需要训练的其实是一个噪声预测器，假设网络输出的噪声为 $\epsilon_\theta(x_t, t)$ ，且损失函数为：

$$\|\epsilon - \epsilon_\theta(x_t, t)\|^2 = \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1 - \bar{\alpha}_t)}\epsilon, t)\|^2 \quad (3-10)$$

3.3.2 改进的 U-Net 模型

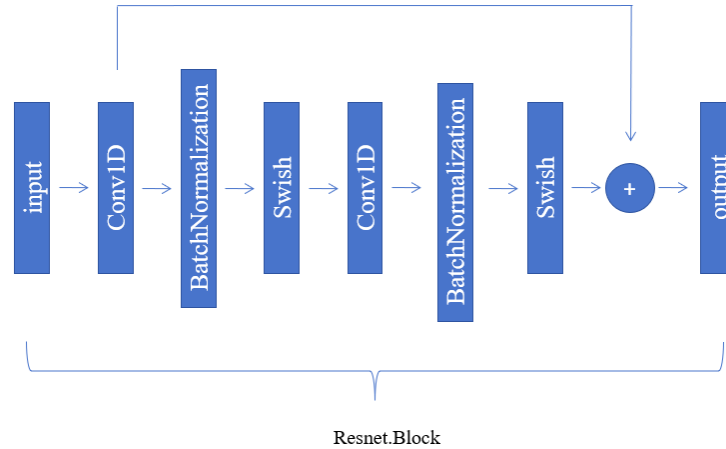


图 3-2 残差块的基本架构

本文使用 U-Net 时基于图上的残差块。

在去噪过程中，使用 U-Net 是类似扩散模型比较常见的做法。其主要的结构分为编码器，中间层，上下采样和解码器部分。为适配语音驱动的三维人脸动画项目，本文创新性地对 U-Net 做了以下修改：

(1) 由于 U-Net 提出时被应用于图像等二维数据，而本文的数据为一维的 Blendshape 参数，因此输出层部分都基于一维卷积神经网络进行训练。

(2) 删除了 U-Net 的上、下采样过程，只由编码器，中间层和解码器组成，其中的卷积模块采用 ResNet 的残差处理方式，原来的 U-Net 在处理图像时，需要通过下采样减少空间维度，增加特征通道以提取更高层次的语义信息，并通过

上采样恢复细节信息，而本文的数据为 Blendshape 参数，不必涉及空间维度的转变，同时也减少了模型大小，提高了预测速度。

(3) 注入条件时相比于使用交叉注意力机制注入，本文使用嵌入表示的方式 (Embeddings) 注入音频特征信息，以提高在高质量小数据集上的性能表现。这一改动允许了 Blendshape 序列的随机裁剪，并能在推理时推广到任意长度的序列。而这一数据增强策略对于基于 transformer 的网络架构时不可能的，因为它们依赖于一致的位置编码。

3.3.3 音频特征条件注入

有一种条件注入的较常见的做法是使用注意力机制注入条件，但本文选择使用嵌入表示的方式注入音频条件。通过使用一维卷积网络作为主干和音频条件注入方式的改变，相比其他方法用伪真值注释在更大的数据集上训练 Transformer 架构，修改后的网络可以提高在小且高质量的数据集上的表现。虽然目前相关的三维视听数据集在逐渐壮大，但高成本高难度的前提仍没有改变，因此在高质量的小数据集上实现更高精度的算法仍在许多具体领域有重要作用，例如动画制作领域，具体的人物制作等等。

3.4 损失函数

本文的损失函数主要参考 Audio2Face 的损失函数，并由于原模型输出的是顶点信息，本文输出的是 Blendshape 参数信息，对损失函数进行一定修改。

损失函数主要分为三部分，位置偏差、连续偏差和正则化项。

3.4.1 位置偏差

这一项为主要项，计算的是每位 Blendshape 参数的预测值和实际值的平方差的平均值：

$$P(x) = \sum_{i=1}^n \left(y^{(i)}(x) - \hat{y}^{(i)}(x) \right)^2 \quad (3-11)$$

其中 n 为 Blendshape 参数的维数（本文中为 37）。

3.4.2 连续偏差

由于本文并没有将 Blendshape 参数序列纳入输入范围，而是单帧进行训练，在一定程度上导致了嘴部抖动，本文的解决方法是将一个 batchsize 的数据前后等分，然后计算每对 Blendshape 参数的差值：

$$M(x) = 2 \sum_{i=1}^{\frac{bs}{2}} \left(m[y^{(i)}(x)] - m[\hat{y}^{(i)}(x)] \right)^2 \quad (3-12)$$

其中 $m(x)$ 表示为两对应帧的差值。

3.4.3 正则化项

这一项损失是为了希望情绪特征尽量地被归结于长期过程，而减小其对短期过程的影响，于上一项类似，可以使用同样的 $m(x)$ 来定义：

$$R'(x) = \frac{2}{E} \sum_{i=1}^E m[e^{(i)}(x)]^2 \quad (3-13)$$

其中 E 为情绪特征向量的维数。再借鉴批量归一化的思想，最终得到损失：

$$R(x) = \frac{R'(x)}{\frac{1}{EB} \sum_{i=1}^E \sum_{j=1}^B e^{(i)} x_j^2} \quad (3-14)$$

其中 B 为 batchsize。

3.5 实验结果与分析

由于语音驱动的人脸动画模型的输出方式多变，有基于 Mesh 序列或基于 Blendshape 参数的不同输出方式，以及人脸模型的 Blendshape 参数构建方式也有较大不同，本文主要与具有相同输出结构的 Audio2Face^[2]网络进行对比。

3.5.1 实验数据集

本文训练、实验都使用 FACEGOOD 开源的数据集。该数据集提供了近 30 万帧的音频和对应人脸表情的 Blendshape 参数。数据集的构建依赖于专业的面部捕捉软件，如 FACEGOOD 的 AVATARY 软件，以下是以 AVATARY 为例的数据构建流程：

(1) 录入音频和使用 AVATARY 的 Tracker 录入对应的人脸至 maya 软件，同时注意音频需要包含丰富的元音以及夸张和正常的说话方式。

(2) 对音频进行 LPC 处理，以 520ms 的窗口进行音频窗口的分割。

(3) 将得到的对应人脸放入 maya 软件中，根据关键帧和指定的 Blendshape 参数导出需要的 37 维关键 Blendshape。至此，就得到了模型需要的视听数据集。

3.5.2 实验设置

实验的对比模型为本文的 Audio2Blendshape 模型和有相同输入输出的 Audio2Face^[2]模型，在上述数据集中使用 Adam 优化器，且学习率为 1e-08 的条件下训练，以 32 的 batchsize 训练 500 个批次，并在 8000 帧的测试集中进行评估。

3.5.3 实验指标

本文使用的实验指标主要是均方误差（MSE，Mean Squared Error）。均方误差是衡量数据精度的一种常用方法，用于计算预测值与实际值之间差异的平方的平均值。其计算公式为：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3-15)$$

此外，本文以一段 3 秒的测试音频为例，对比模型的预测耗时。

3.5.4 实验分析

表 3-1 模型性能评估对比表

method	MSE	time-consuming (ms)
Audio2Blendshape	0.00083	2.348
Audio2Face	0.00127	1.995

如表 3-1 所示，本文使用的对比网络 Audio2Face^[2]同样基于情绪潜代码但其骨干网络为深度神经网络。同样的训练批次（500）情况下，以 3 秒钟的音频为输入条件，推演时间虽然从 1.995 毫秒上升到了 2.348 毫秒，但平均平方误差从

0.00127 降低到了 0.00083。

3.6 本章小结

本章主要介绍了本文使用的模型，包括模型的具体框架以及条件扩散模型和情绪潜代码的具体使用和优势，最后介绍了模型的实验结果。

4. 系统设计与实现

4.1 需求分析

4.1.1 功能需求

- (1) 传入音频文件，页面展示对应的三维人脸动画。
- (2) 支持静态查看当前的标准人脸模型，并支持替换使用相同 Blendshape 参数构建方式的人脸模型。
- (3) 查看当前 Blendshape 参数的构建名称。
- (4) 后端支持使用预训练好的模型和输入数据返回预测结果。

4.1.2 界面需求

主要在 pc 端进行实现，大致页面包括动画展示页面，静态标准人脸模型查看及修改页面和 Blendshape 参数查看页面。要求界面清晰美观，操作方便。

4.2 系统模块设计

主要的模块设计及其具体工作如下：

- (1) 音频和动画模块：支持上传音频数据文件，文件格式为 wav 格式。在输入音频后，载入标准人脸模型，并从后端模型预测 Blendshape 参数序列，得到结果后在前端与标准人脸模型结合得到流畅的人脸动画。
- (2) 展示和更换模型模块：展示目前的静态标准人脸模型，支持鼠标控制拉近人脸或控制视角观察，同时支持更换人脸模型。
- (3) Blendshape 参数展示模块：展示人脸的 Blendshape 参数名称，并指明哪些参数为较为关键，也就是实际控制变化的参数。
- (4) 模型预测模块：后端对输入的音频文件进行预处理后，使用预训练好的模型进行预测，得到目标 Blendshape 参数序列。

4.3 系统流程

系统主要由三个界面组成，分别为音频输入和动画展示页面，静态人脸模型查看和替换界面和 Blendshape 参数查看界面，界面支持侧边栏点击跳转，以下是用户使用该系统的具体流程：

- (1) 用户进入系统后，默认进入音频输入和动画展示模块，根据提示上传音频文件后自动渲染生成对应的人脸动画，同时模型动画支持鼠标操作，拉近或调整视角等等。
- (2) 进入静态人脸模型查看和替换界面时，自动渲染当前静态的标准人脸模型，并同样支持鼠标操作，拉近或调整视角等等。点击更换按钮，选择需要更换的人脸模型，更换后重新加载静态的标准人脸模型。（要想正常运转，必须上传以同种方

式构建 Blendshape 参数的人脸模型)

(3) 进入 Blendshape 参数展示模块, 根据预设定好的数组展示当前静态人脸模型的 Blendshape 参数构建名称 (116 维), 同时支持查看当前人脸动画使用的相关 Blendshape 参数名称 (37 维), 这 37 维 Blendshape 参数为模型的最终输出和模型动画渲染时更新的参数。

4.4 前端页面实现

本文的前端页面工作主要包括音频文件上传组件, 模型展示组件和界面样式组件。

4.4.1 音频文件上传

XMLHttpRequest (XHR) 是在 JavaScript 中进行 HTTP 请求的技术, ajax 对其进行了高效的封装, 而进一步地, axios 在 ajax 的基础上又添加了 promise 的异步机制。这一点在具体应用时是非常有用的。例如, 本文希望在用户上传音频文件后才开始 Three.js 的初始化, 故可以直接在 axios 命令后使用 then 函数直接执行拿到数据后的操作, 让代码编写更加符合逻辑。

上传文件时使用 FormData 对象, 它原本是用于创建表单数据的 JavaScript 对象, 提供了一种简单方式来构建包含键值对的表单数据, 可以包含文本字段和文件字段等等。

通过 append 方法将音频文件推入 FormData 中上传时, 需要在 “headers” 设置 “‘Content-Type’: ‘multipart/form-data’”。该字段表明允许请求体包含多个部分, 每个部分都有自己的内容类型和数据, 例如文件和文本共存的情况。不过, 就算表单中只有文件上传时, 仍需要设置该字段, 才能让服务器正确解析请求体中的文件数据。

4.4.2 静态人脸模型展示



图 4-1 Three.js 渲染模型的基本过程

Three.js 是运行在浏览器中的 3D 引擎, 是用 JavaScript 编写的 WebGL 第三方库。提供了非常多的三维模型展示功能。开发者可以用它创建各种三维场景, 包括了摄影机、光影、材质等各种对象。它的主要优势之一就是简化了 WebGL 的复杂性, 让开发者可以避开 WebGL 的大部分底层细节, 使用简洁的接口来处理场景图、照明、纹理、材料、摄像头控制和动画等等。Three.js 的核心三要素为场景 (Scene), 相机 (Camera) 和渲染器 (Renderer)。使用 Three.js 展示三维模型的基本流程如下:

(1) 创建一个场景对象作为其他模型的容器；

(2) 创建一个相机，以控制用户视角。相机分为透视相机和正交相机，其中前者适用于大多 3D 场景。此外，还有许多特殊功能，包括处理用户输入，响应窗口尺寸变化等等；

(3) 创建一个渲染器，负责将场景渲染至<canvas>元素，最常用的是基于 WebGL API 的“WebGLRenderer”，支持修改属性进行性能优化，包括提高边缘平滑度，控制精度，选择性能和支持响应式更新。最后，渲染器需要在动画循环（animate）中调用，以更新场景变化。

(4) 添加光源，背景等元素丰富场景；光源是任何 3D 场景的核心组件之一，使材质，纹理和其他细节能够生动地呈现。光源有无方向的环境光，从点发射的点光源，光线彼此平行的平行光等等。往往需要结合多种光源才能合成真实自然的场景。同时阴影的合理使用也可以较好提高真实性。

(5) 通过动画循环不断更新场景，以响应输入或动画效果。动画渲染是用于不断更新场景的重要机制，通过“requestAnimationFrame”实现。基本的动画循环主要包括更新相机，物体和动画，渲染场景，最后重复调用动画循环。动画循环往往涉及到性能问题。适当地选择渲染对象，避免创建更多的临时对象可以有效减少性能消耗。特别地，Three.js 提供了一个“Clock”工具类，来管理与时间相关的动画控制。例如特定帧率或控制帧率最大值以提高性能的动画等等。



图 4-2 人脸模型展示



图 4-3 调整人脸模型角度

Three.js 支持多种三维模型的导入，不同格式都有其特点和应用场景。最原始的是 JSON 格式，加载最快但文件往往较大。obj，fbx 等工业标准的模型格式能支

持复杂的建模细节，但在解析时相对困难。目前前端领域用的最多的三维模型是 `glTF` 格式，虽然在细节和一些复杂的特性支持上不如专业的三维建模软件，但其快速加载的特性和描述信息的完整性仍旧让它成为最优秀的建模格式之一。

本文支持 `glTF` 模型的导入，相比于不支持动画和场景的 `stl`, `obj` 等静态模型，这该模型格式不仅包含了几何、材质信息，也可以存储动画等数据，可以通过 `animations` 属性查看。它在读取速度较快的同时，也能在模型格式转换时保留更多层次的信息。

由于人脸模型本身的复杂性和具体场景的需求不同，`Blendshape` 的构建方式也有比较大的差异。根据追求真实性的程度高低，表情的覆盖范围以及动画展示的精确程度，`Blendshape` 的构建维度也从二十维到上百维不等。在语音驱动的三维人脸动画这一主题下，为实现自然的人脸动画，对 `Blendshape` 维数的构建要求是相当高的。但在追求精度的同时，也要考虑实际展示平台的渲染压力，否则会导致动画卡顿。本文在使用 116 维的 `Blendshape` 参数构建标准人脸模型的同时，选择了 37 维较关键的参数进行动画更新，以谋求表现力和资源效率之间的平衡。

4.4.3 动态人脸动画实现

本文以 30 帧每秒的帧数进行动画渲染，同时对场景进行一定程度上的美化。由于 `animate` 渲染时默认为 60 帧每秒，本文做了特别处理以控制帧数，主要的具体流程如下：

(1) 创建场景，相机，渲染器。相机设置上，使用远景相机，相机的横纵比设置为屏幕宽高比，否则会出现挤压变形。这一步时为了显示动画效果，先使用 `requestAnimationFrame` 创建了动画循环函数。

(2) 导入标准人脸模型。调用模型文件对应的 `loader` 载入模型，添加纹理后发现人脸已经出现，但是过于模糊。其原因主要是设备的物理像素分辨率和 `CSS` 像素分辨率的比值问题，绘制出的图像由于高清屏物理设备的影响变大，而渲染窗口没有变化，就会导致图像挤压缩放，最终导致模糊问题。解决方法就是调用 `devicePixelRatio` 属性判断比值大小，在需要调整时重新设置渲染器的窗口大小。

(3) 设置 `frameTime`（每帧持续时间），`clock`（钟表对象），`timeS`（已执行时间）用于特定帧率控制。

(4) 在调用 `animate` 时，进入函数通过 `clock.getDelta()` 方法得到两帧之间的时间间隔，并加至 `timeS`，比较 `timeS` 和 `frameTime` 的大小，如果到时间则渲染下一帧，没到时间就跳过。

(5) 添加轨道控制器，到目前为止已经可以显示较流畅的人脸动画，但视角固定，故希望能根据鼠标操作，拉近或调整标准人脸模型的观察视角。引入 `OrbitControls`，同时添加 `enableDamping` 属性添加阻尼感，提高真实性，最后在渲染函数前调用即可。

(6) 添加光影效果，合适的光照可以进一步提高场景的真实性。本文添加了

平行光和半球光。平行光从一个特定的方向照射，一般用来模拟太阳光，可以产生阴影；半球光用来模拟环境光，整个场景都会被均匀地照亮，因此可以为整个场景提供一个基础的照明效果。为支持阴影，要为渲染器设置 `shadowMap.enabled`，同时为模型的每个部分添加材质时，让其能产生阴影。最后为背景添加雾化效果，用于增加场景的立体感，营造更加真实的环境氛围。

4.4.4 动画渲染问题处理

本文在具体实现时，发现在具体动画渲染时，会发生“白边闪现”的情况，同时也没有生成正确动画。Three.js 在渲染多边形几何体时通常都会自动将其转换成三角形以方便渲染。其原因主要有以下几点：1. 三角形是计算机图形学中的基本构建单位，因此许多渲染器都是基于三角形；2. 三角形的表面法向量，纹理坐标或光照等属性都比较易于计算；3. 任何多边形都可以被分解成三角形，因此在转化时几乎可以不损失具体的几何形状。但具体到复杂的人脸模型，特别是人脸动画问题时，法线方向在变形动画时会出现法线不连续，不平滑，导致动画出现“白边闪现”的问题。

本文采用的原始模型类型为 ma 模型，在转成 Three.js 支持的 fbx 格式时出现了上述问题，最终解决方法是在导出时勾选“三角化”选项，直接将模型转换成三角形格式进行渲染，而不是依赖 Three.js 的自动三角化功能。

4.4.5 页面样式搭建

本文主要使用 Element Plus 框架进行样式搭建。Element Plus 是基于 Vue 3 的 UI 组件库，包含了非常丰富的扩展组件，可以直接提供页面的大体布局结构，表格表单的构建到各种图标，按钮，通知弹窗等等。对于大部分的基础网页，它可以提供绝大多数美观高效的功能组件。不光如此，Element Plus 提供了详细的文档和示例教程，让各种复杂的 UI 组件可以真正地“开箱即用”，是基于 Vue 技术栈的前端开发者们必备的工具之一。

本文的基本页面构建如下图：

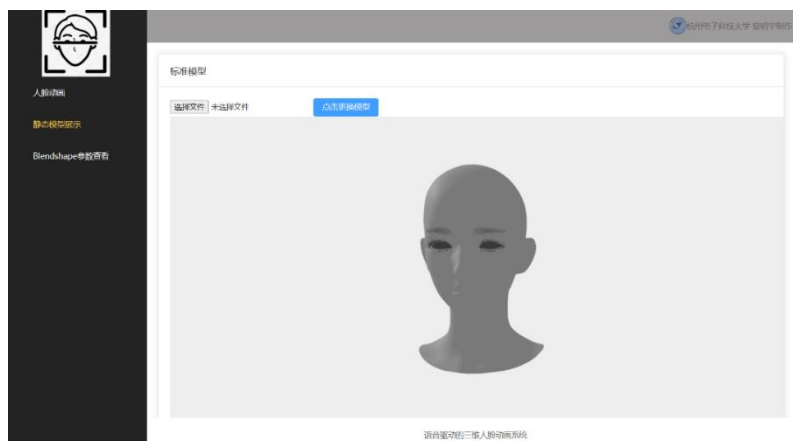


图 4-4 页面布局

使用 el-container 组件包裹整个页面的同时，头部，侧边栏和主体区域分别由 el-header, el-aside 和 el-main 包裹。el-aside 中使用 el-menu 来包裹牡蛎，el-menu-item 来包裹具体的导航项，通过为 el-menu 添加 router 属性后，即可在其中的子标签 el-menu-item 中使用 index 标签直接进行路由跳转。el-main 中使用子组件 page-container 来封装不同子界面的具体样式。在具体页面上，使用 el-card 包裹具体的页面内容，可以自动生成上图表现的边框阴影，增加界面的美观程度。

特别地，Blendshape 参数展示界面如下：

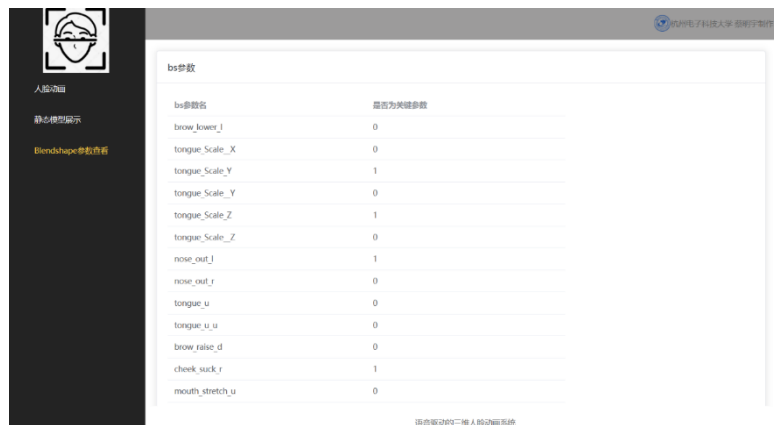


图 4-5 Blendshape 参数展示界面

使用 el-table 组件封装具体的列表。主要有 bs 参数名和是否为关键参数两个字段，这里的关键字段就是指该参数是否是人脸动画更新的参数，这里使用数组循环赋值的方式赋值该字段，所以可以使用更改数组的方式替换使用的关键参数，不过需要配合算法模型的修改。

4.4.6 图标、弹窗美化

本文主要使用了图标（Icon），按钮（Button）和警告（Alert）替换其原本的内容。注意导入时可以选择两种方案：一是在 main.js 中全局引入，二是使用 babel-plugin-component 插件后，就可以在对应界面添加目标组件。以下是一些使用范例。



图 4-6 图标样式

4.4.7 静态模型修改应用至动画

由于在静态模型展示界面更换模型时希望能将模型同样应用至动画界面。这里使用 Pinia 进行状态管理。

Pinia 是一个 Vue 框架专属的状态管理库，是 Vuex 的代替者。Pinia 使用 Store 来管理应用状态，可以用来管理需要在页面组件之间来回交互的共有数据。相比于

Vuex, Pinia 的 API 更加灵活简便, 让状态管理更加通俗易懂。

在本文中, 原先想要使用的方法是后端一并返回 Blendshape 参数和标准的人脸模型, 但由于标准人脸模型的数据大小较大, 最后决定先在本地导入标准人脸模型。与此同时又产生了新的问题, Three.js 框架中的 loader 不支持直接访问文件系统, 本文的做法是使用 Vue CLI 的特性, 它在目录生成时自带 “public” 文件夹, 将需要替换的模型放置该目录下, 就可直接通过公共路径直接访问。因此, 本文在替换模型时, 在使用 input 元素获取文件后, 使用 event.target.files.name 获取选中的文件名, 并手动添加 “./” 后, 将其添加到 Pinia 仓库中, 用 Three.js 的 loader 加载模型后, 便可以正常更换。同时在替换后, 不能简单再次调用三维场景的创捷函数 initThree, 而是需要首先调用 scene.remove(model), 不然就会因为在渲染时没有更新具体的模型而报错。

4.5 后端模型部署

4.5.1 部署模型概述

部署模型到生产环境中作为真正把算法应用于生产是非常重要的一环。而在实际场景中, 在时刻变化的需求, 无限增长的数据量, 迅速发展的计算机资源和越来越复杂的模型这一系列影响因素下, 如何保障服务的稳定, 泛用和实时性, 成为越来越重要的问题。模型部署主要涉及到存储模型, 配置模型, 处理数据流程等诸多流程。

模型存储和导入作为第一步, 需要针对具体的场景选择合适的格式。就 TensorFlow 来说, SavedModel 格式是标准的模型保存格式, 支持 TensorFlow 一系列服务, 集成性较好, 并支持跨平台部署; HDF5 格式因其较小的文件, 通常适合在本地或云端部署; 对应移动, 嵌入式设备和浏览器平台, 分别提出了 TensorFlow Lite 和 TensorFlow.js 格式。

合适的模型部署架构也至关重要。具体有以下几种: 1. 在后端实现 RESTful API, 客户端通过 HTTP 请求发送数据, 后端处理并调用模型预测, 最后返回结果, 具有较好的灵活性; 2. 将模型部署为微服务, 通过消息队列等方式进行通信, 适用于大规模应用。3. 将模型部署至无服务器平台, 适用于轻量级, 低负载的应用场景; 4. 使用专门的模型服务器, 具有高性能和高成本的特点; 5. 把模型部署到边缘设备, 适用于离线预测或实时推理等类似场景。

本文暂不考虑复杂的生产环境, 使用 Django 框架直接部署模型, 主要流程包括音频处理, 模型预测和结果返回部分。

4.5.2 模型具体部署流程

本文的后端模型部署主要包含以下步骤:

(1) 音频处理。首先后端得到前端传来的 wav 文件后, 提取音频的采样率 (rate) 和信号 (signal), 并将音频分割至 520ms, 把每段音频窗口传入 LPC 函数

处理，得到符合要求的模型输入数据。本文在实际操作时发现由于前端以二进制形式传递 wav 文件，在后端直接提取信号和采样率时会导致运行出错，这里先将信息重新传入临时的 wav 文件，采用 `scipy.io.wavfile` 模块中的 `wavfile` 函数对其进行信号提取后，再删除临时文件。

(2) 模型预测。首先在模型保存时转成 `tflite` 格式，以未来支持移动端或嵌入式设备。后端使用 `tf.lite.Interpreter()` 方法传入训练好的模型和处理好的输入数据，并进行预测。

(3) 结果返回。模型返回音频对应的 `Blendshape` 参数序列，对应的视频帧率为 30 帧每秒。例如，传入 3 秒的音频文件，最后得到的结果反馈为 `90*37` 维的 `Blendshape` 参数序列。

4.5.3 跨域问题解决

在前端的 API 端口配置完成，后端服务器运行后，项目运行时会报跨域错误。跨域问题主要发生在一个域下的网页去请求另一个域下的资源时会受到限制。其根本原因是浏览器的同源策略。没有特殊配置的情况下，不允许源之间的资源传递，而源指的是域，协议和端口的组合，即使都部署在本地运行，也会因为端口不一致导致跨域问题。

比较常用的解决方法是使用跨域资源共享机制。具体而言就是在 HTTP 响应头部添加“`Access-Control-Allow-Origin`”字段来指定允许跨域的域名。另外还有用服务器代理的方式，在本地创建一个虚拟服务器，同时代理前端请求和服务器响应，这样在前端和服务器之间进行数据交互时就能够避免跨域问题的产生。在早期还有用 JSONP，利用 `<script>` 表情不受同源策略限制的特性进行网络请求的解决方法。虽然使用简单，但因为安全风险大，只支持 GET 请求等弊端，渐渐退出历史舞台。

本文使用跨域资源共享机制解决，使用 Django 框架时可以使用第三方扩展插件“`django-cors-headers`”，注意在中间件处理使用时放在第一条，以保证第一时间处理。

4.6 系统测试

系统测试是软件开发过程中的一个重要阶段，也是最后的阶段，主要目的是确保整个系统满足规定的需求并能在各种环境下正确运行。本文主要对系统的实现功能进行测试。

系统应有功能和对应验证有以下几个方面：

- (1) 验证用户是否能够上传音频文件，并符合格式要求。
- (2) 验证后端模型是否能按要求进行数据处理，并返回结果。
- (3) 验证前端是否能根据模型预测结果和标准人脸模型生成人脸动画。
- (4) 验证是否能在静态模型界面替换标准人脸模型，并一同更改至动画界面。
- (5) 验证是否能正确展示 `Blendshape` 参数的名称和构成。

经过多次测试和调整，本文提出的系统成功通过以上验证，达成了系统功能的基本完善。

系统功能的测试表如下：

表 4-1 系统功能测试表

测试模块	预期结果	实际结果	测试结论	测试模块
音频文件上传	后端成功接收到音频数据	后端成功接收到音频数据	通过	音频文件上传
模型预测	成功返回 Blendshape 序列	成功返回 Blendshape 序列	通过	模型预测
人脸动画生成	成功生成人脸动画	成功生成人脸动画	通过	人脸动画生成
人脸模型切换	成功切换标准模型	成功切换标准模型	通过	人脸模型切换
Blendshape 参数展示	成功展示 Blendshape 参数	成功展示 Blendshape 参数	通过	Blendshape 参数展示

4.6 本章小结

本章主要介绍了本文的系统设计以及实现。包括系统的大致框架、用户使用流程，以及各模块，包括前端、后端模块的主要功能和具体实现，以及系统的功能测试。

5. 总结与展望

5.1 工作总结

本文在广泛研究语音驱动的三维人脸动画模型算法的基础上，融合相关算法的优点，提出了基于情绪潜代码和条件扩散模型的算法。该算法使用情绪潜代码的思想避免了语音特征的复杂处理，支持从新音频中提取出情绪特征，提高了模型的推断速度；同时引入条件扩散模型以提高模型的细节表现；也对条件扩散模型做了一定修改，以提高在高质量小数据集上的性能表现。

在系统实现方面，本文使用前端 Vue 框架和后端 Django 框架，在页面端实现了较流畅的三维人脸动画展示，并支持标准人脸模型的替换。实现了特定帧率（30 帧每秒）的动画演示。

5.2 不足和未来展望

本文还有诸多不足和可以发展的未来方向，主要包含以下几个方面：

（1）虽然修改后条件扩散模型提高了在高质量小数据集上的表现，但仍受限于小数据集，该算法对不同人声的泛用性能力较弱，将来可以通过扩大数据集或引入身份特征进行优化。

（2）本文的情绪潜代码使用潜力还未开发完全，只支持新音频的情绪推断，而未能支持辅助输入，将来通过添加对情绪潜代码的提取和输入支持，以提高使用的灵活性。

（3）在系统实现方面，虽然动画较流畅，但是由于人脸模型的复杂性，模型加载速度较慢，同时动画做不到与损失一致的精准性，需要未来使用新的技术或策略加载人脸模型，以提高用户体验。

在本文的研究过程中，我对人工智能算法使用有了更深的理解，完成了从模型选择、优化、训练、保存、加载和部署过程。同时尝试了网页端的三维模型动画实现。在未来，我将继续深入学习三维模型的网页端实现和相关算法，了解更多算法模型和三维渲染方案。

致谢

在几个月的时间里，巨大的就业压力下，从论文选题、模型优化、系统制作到论文撰写，一度有过疲惫痛苦的时期，但最终得以开花结果，这其中不乏自己的努力，也离不开同学和老师对我的帮助。

感谢张聪老师在百忙之中，帮助我确定选题，并对我的设计思路和系统实现提供恰当有效的建议。这一系列帮助我都铭记于心。

感谢林菲老师在四年里对我的指导和照顾，指导我在人工智能方面的学习和进步。在学习算法的过程中，不仅养成了严谨的学习习惯，更学会了如何生活，如何做人。

感谢大学四年的朋友和同学们与我一同进步，同甘共苦。

感谢母校和老师们在大学四年中对我的培养。

参考文献

- [1] 刘贤梅, 刘露, 贾迪, 赵娅, 田枫. 基于语音驱动的三维人脸动画技术综述[J]. 计算机系统应用, 2022, 31 (10): 44-50.
- [2] Karras T, Aila T, Laine S, et al. Audio-driven facial animation by joint end-to-end learning of pose and emotion[J]. ACM Transactions on Graphics (TOG), 2017, 36(4): 1-12.
- [3] Cudeiro D, Bolkart T, Laidlaw C, et al. Capture, learning, and synthesis of 3D speaking styles[C]. Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019: 10101-10111.
- [4] Thambiraja B, Habibie I, Aliakbarian S, et al. Imitator: Personalized speech-driven 3d facial animation[C]. Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 20621-20631.
- [5] Fan Y, Lin Z, Saito J, et al. Faceformer: Speech-driven 3d facial animation with transformers[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 18770-18780.
- [6] Xing J, Xia M, Zhang Y, et al. Codetalker: Speech-driven 3d facial animation with discrete motion prior[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 12780-12790.
- [7] Cheng K, Cun X, Zhang Y, et al. Videoretalking: Audio-based lip synchronization for talking head video editing in the wild[C]. SIGGRAPH Asia 2022 Conference Papers. 2022: 1-9.
- [8] Peng Z, Wu H, Song Z, et al. Emotalk: Speech-driven emotional disentanglement for 3d face animation[C]. Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 20687-20697.
- [9] Wu H, Zhou S, Jia J, et al. Speech-Driven 3D Face Animation with Composite and Regional Facial Movements[C]. Proceedings of the 31st ACM International Conference on Multimedia. 2023: 6822-6830.
- [10] He S, He H, Yang S, et al. Speech4Mesh: Speech-Assisted Monocular 3D Facial Reconstruction for Speech-Driven 3D Facial Animation[C]. Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 14192-14202.
- [11] Kwar B, Zada S, Lang O, et al. Imagic: Text-based real image editing with diffusion

- models[C].Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 6007-6017.
- [12] Hertz A, Mokady R, Tenenbaum J, et al. Prompt-to-prompt image editing with cross attention control[J]. arXiv preprint arXiv:2208.01626, 2022.
- [13] Parmar G, Kumar Singh K, Zhang R, et al. Zero-shot image-to-image translation[C].ACM SIGGRAPH 2023 Conference Proceedings. 2023: 1-11.
- [14] Stan S, Haque K I, Yumak Z. Facediffuser: Speech-driven 3d facial animation synthesis using diffusion[C].Proceedings of the 16th ACM SIGGRAPH Conference on Motion, Interaction and Games. 2023: 1-11.
- [15] Thambiraja B, Aliakbarian S, Cosker D, et al. 3diface: Diffusion-based speech-driven 3d facial animation and editing[J].arXiv preprint arXiv:2312.00870, 2023.